AOBP

Centralized SSO, Authentication and Authorization

AOBP — Centralized SSO, Authentication and Authorization

1. Objective

The objective is to establish a single, centralized authentication and authorization architecture for the Airport Operations Business Platform (AOBP).

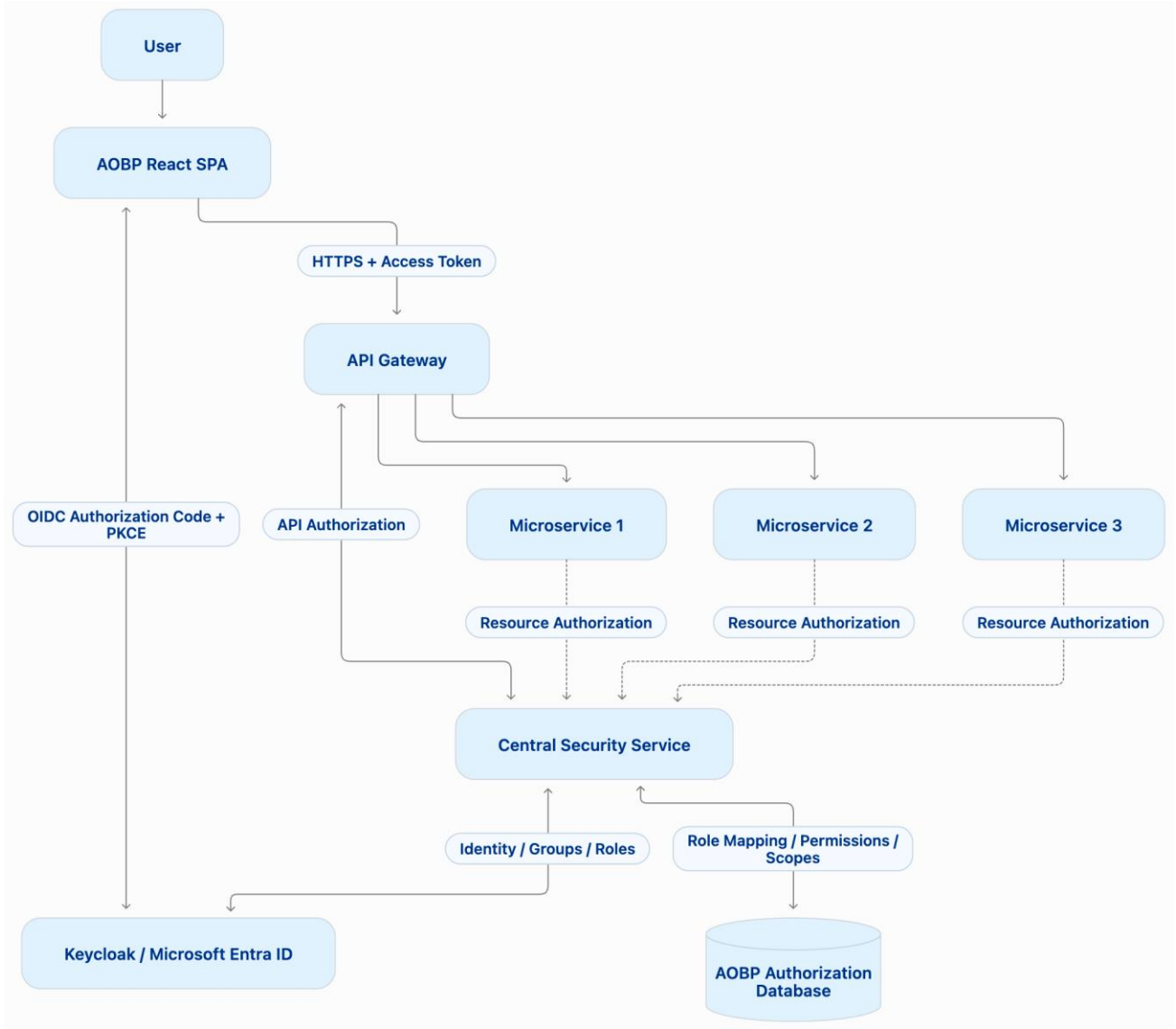
The main architectural principle is:

AOBP will not store or maintain local user profiles. BAC SSO / Identity Provider owns user identity, user lifecycle, user attributes, group membership and authentication. AOBP stores only authorization configuration: SSO group/role mapping, AOBP roles, permissions, policies and audit records.

2. High-level architecture

The recommended design is based on five principles:

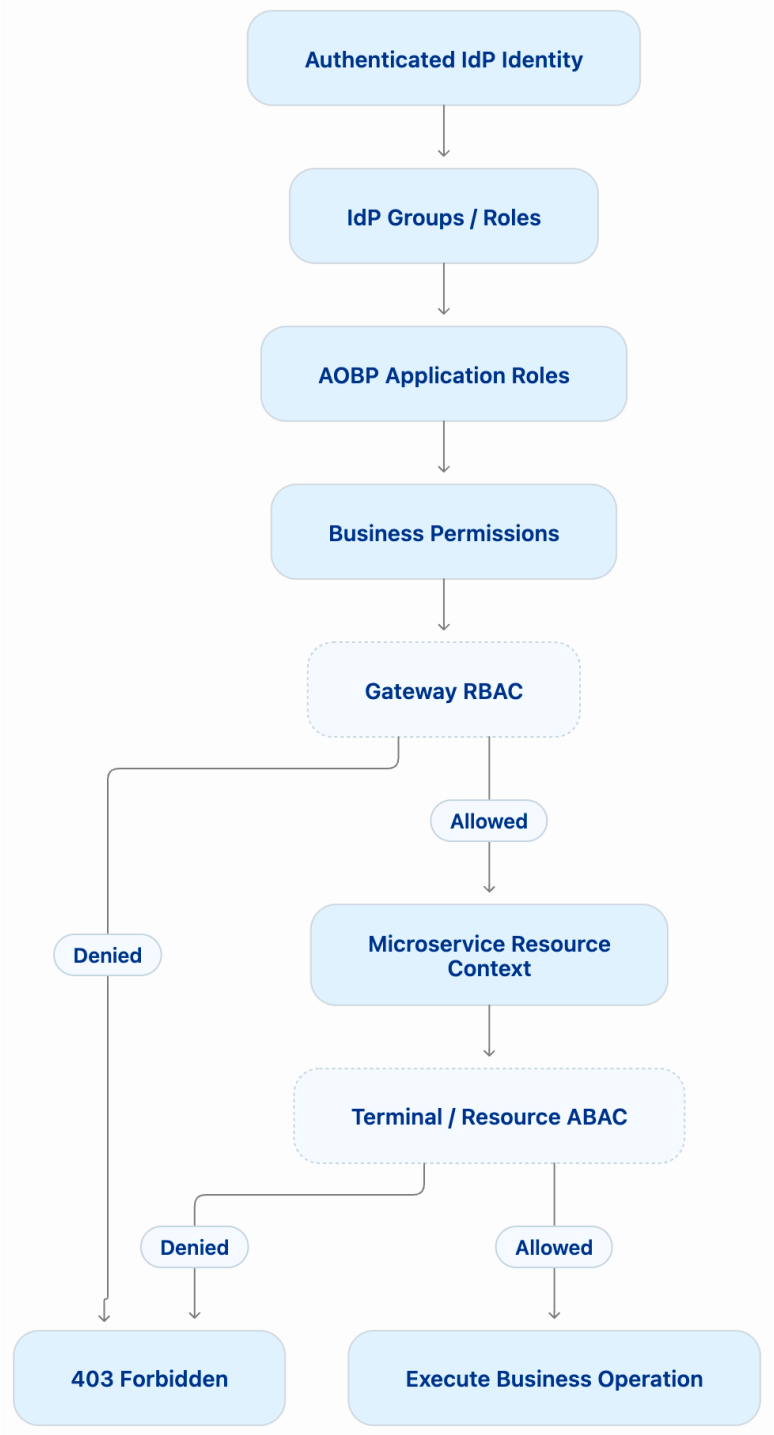
- One application: AOBP does not need application or microservice registration tables for authorization.
- Centralized identity integration: Keycloak or Entra ID owns authentication and external user identities.
- Business-oriented RBAC: AOBP roles grant stable business permissions independent of backend service boundaries.
- Scope-aware ABAC: Data access and other restrictions are evaluated using trusted user and resource attributes.
- Deployment-independent enforcement: The same authorization decisions apply with or without an Istio service mesh.



Component	Responsibility
Identity Provider	Authenticate users, manage users, groups, roles and identity attributes
React SPA	Initiate SSO, maintain authenticated browser session and invoke APIs
API Gateway	Enforce operation-level RBAC and route requests
Central Security Service	Validate tokens, map roles and evaluate RBAC/ABAC
Authorization Database	Store application roles, permissions and mappings
Microservices	Execute business functions and enforce resource-level authorization

3. Implementation Approach

Authorization Flow



Step-1: Register AOBP React SPA in the Identity Provider

AOBP must first be registered as an OIDC client in Keycloak or Microsoft Entra ID.Registration establishes which application may initiate authentication, where the IdP may redirect the browser and which API permissions the client may request.

Step-2: Register the AOBP Protected API

The backend API must also be represented in the IdP as a protected API resource or audience.

Example

Property	Example
API identifier	`aobp-api`
Delegated scope	`aobp-api/access`
Authorized client	`aobp-spa`
Token audience	Provider-specific identifier for the AOBP API

This is a logical API resource, not a registration for every microservice.

Step 3 — Configure IdP groups and roles

The IdP owns the user identity and external group membership.A user may belong to multiple groups. The Central Security Service resolves group membership from trusted token claims or approved IdP APIs.

Step 4 — Define AOBP roles

AOBP roles represent business responsibilities rather than microservice boundaries.

Example

Role	Description
`ADMIN`	Platform administration
`MANAGER`	Management operations
`OPERATOR`	Standard operations
`VIEWER`	Read-only functions

These roles are centrally managed in the AOBP Authorization Database.

Step 5 — Map IdP groups to AOBP roles

The Central Security Service maps trusted external groups or roles to AOBP roles.The mapping table stores the IdP issuer, external entitlement type and stable external entitlement identifier.

Example Mapping

IdP Group	AOBP Role
AOBP_OPERATIONS	OPERATOR
AOBP_MANAGERS	MANAGER
AOBP_VIEWERS	VIEWER

AOBP_ADMINISTRATORS	ADMIN

Step 6 — Define business permissions

Permissions represent actions on AOBP business capabilities.A permission can be used by multiple APIs or microservices without duplication.

Step 7 — Assign permissions to roles (RBAC)

Permissions represent actions on business capabilities.For example, an `OPERATOR` can update data because the role grants `DATA.UPDATE`.

Step 8 - Attribute-Based Access Control(ABAC)

AOBP must support users who have the same role but different data-access scopes. The role determines permitted actions

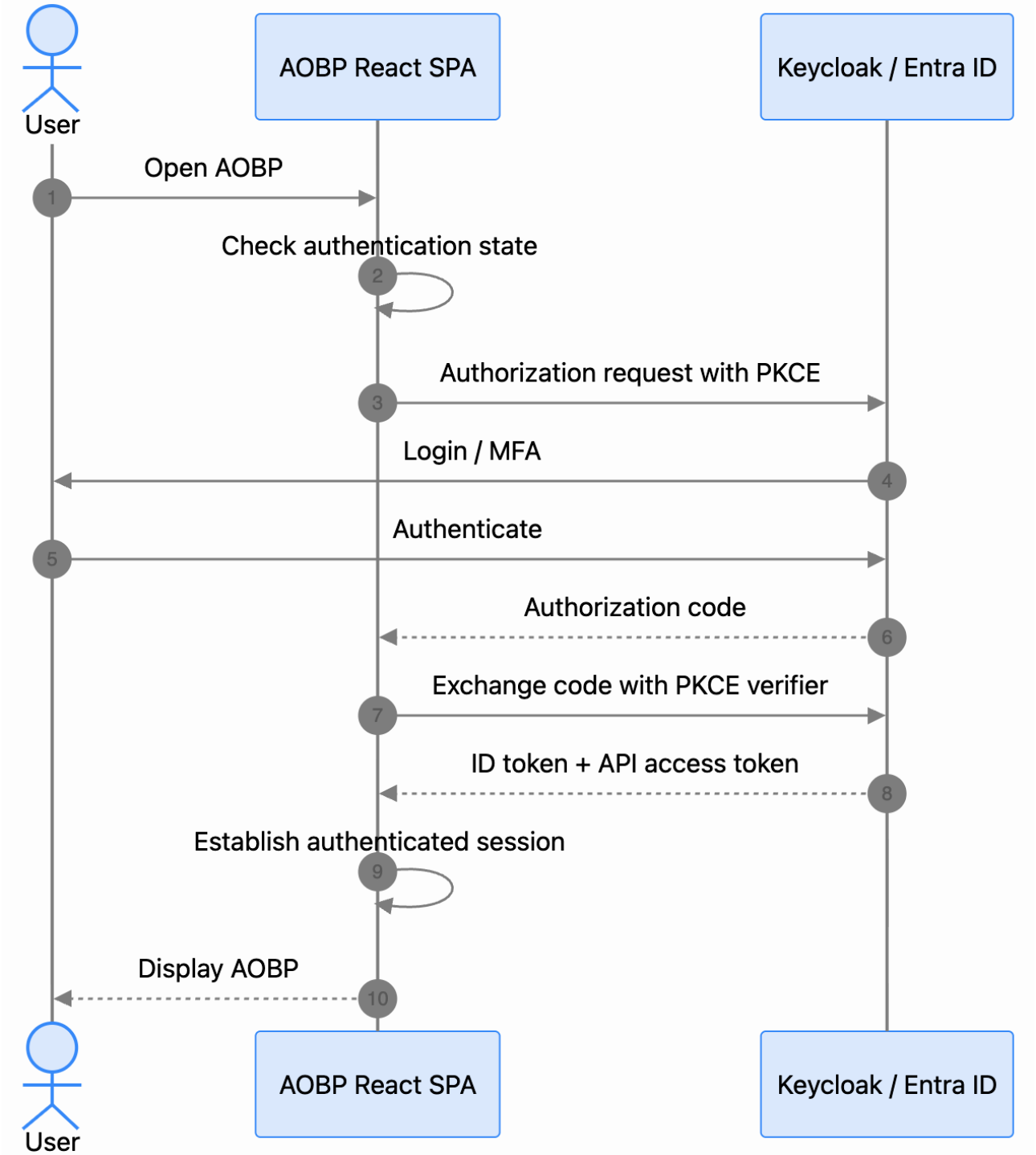
Example:

Users may have identical roles but different terminal access.

User	Role	Allowed Terminal
User A	OPERATOR	T1
User B	OPERATOR	T2
User C	OPERATOR	T1, T2

Authentication Runtime

The React SPA detects that no authenticated session exists and initiates OIDC login.



The IdP authenticates the user and issues tokens.

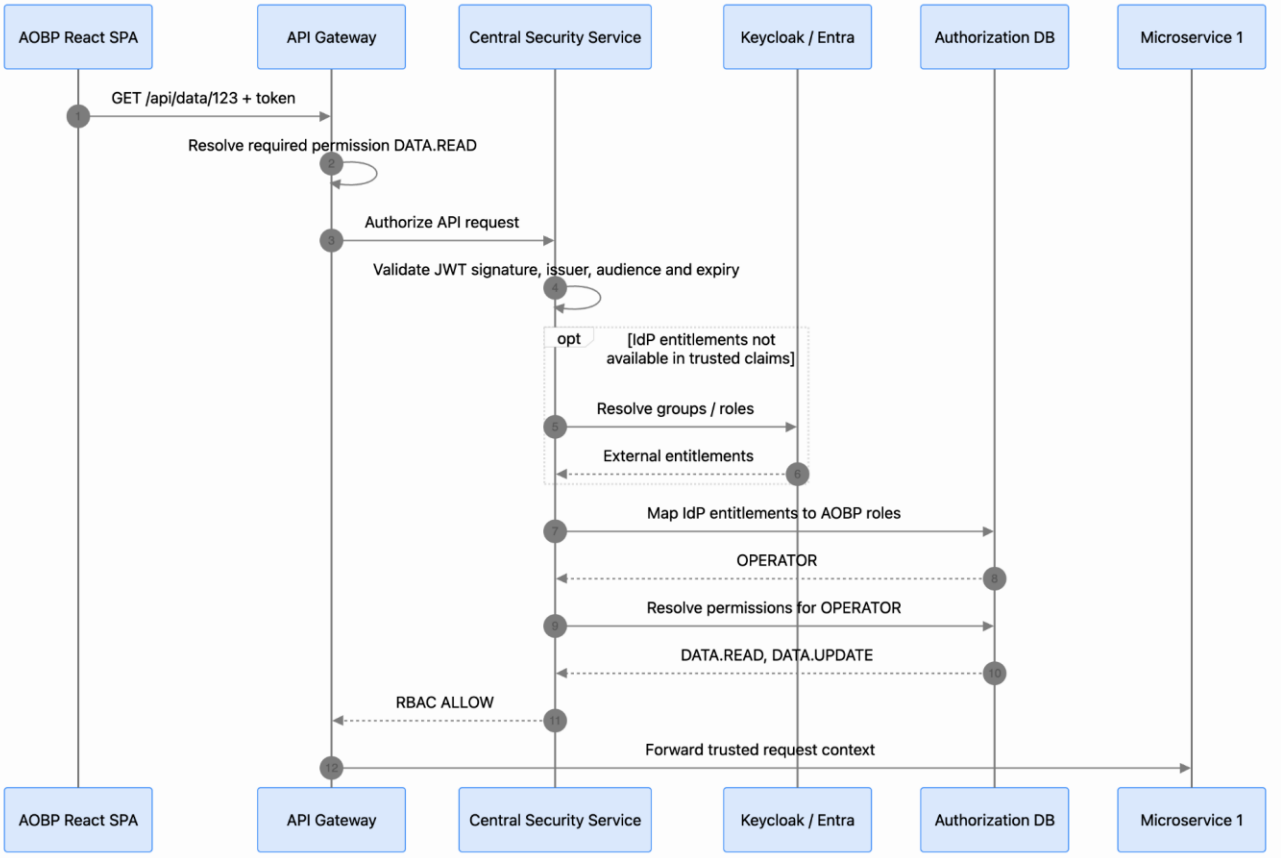
The SPA should keep access tokens in memory where practical, avoid persistent browser storage of bearer tokens and protect against XSS. Refresh-token use depends on IdP configuration and must follow current rotation, lifetime and revocation policies.

Gateway RBAC Authorization

The gateway allows the API request only after the Central Security Service confirms the required permission.

The gateway must not accept permission codes or identity claims directly from untrusted browser headers.

Central Security Service evaluates RBAC



Exmaple:

RBAC Evaluation Process:

- Validate JWT
- Validate issuer
- Validate audience
- Validate expiry
- Resolve IdP groups
- Map groups to AOBP roles
- Load permissions
- Evaluate required permission
- Allow or deny request

Resource-Level ABAC Authorization:

Example:

User:

Allowed Terminal = T1

Requested Resource:

Terminal = T2

Decision:

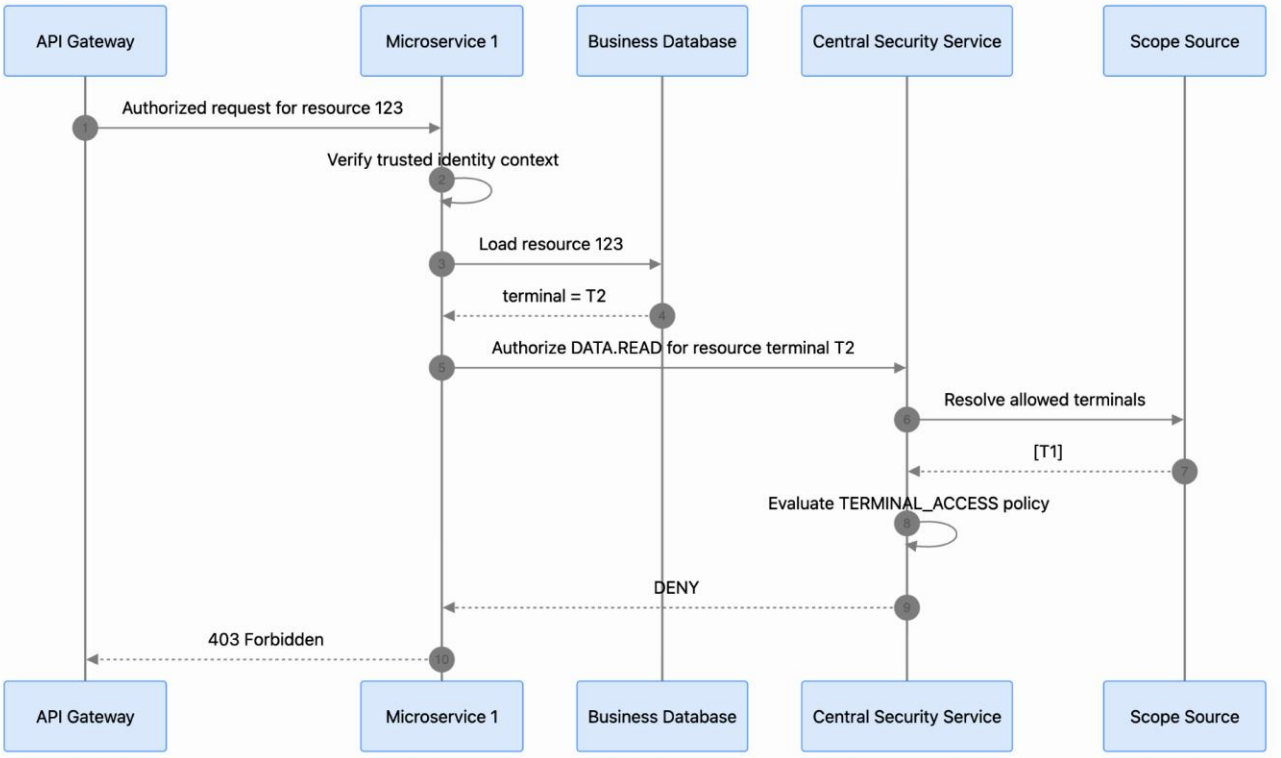
RBAC = ALLOW

ABAC = DENY

Final Decision = DENY

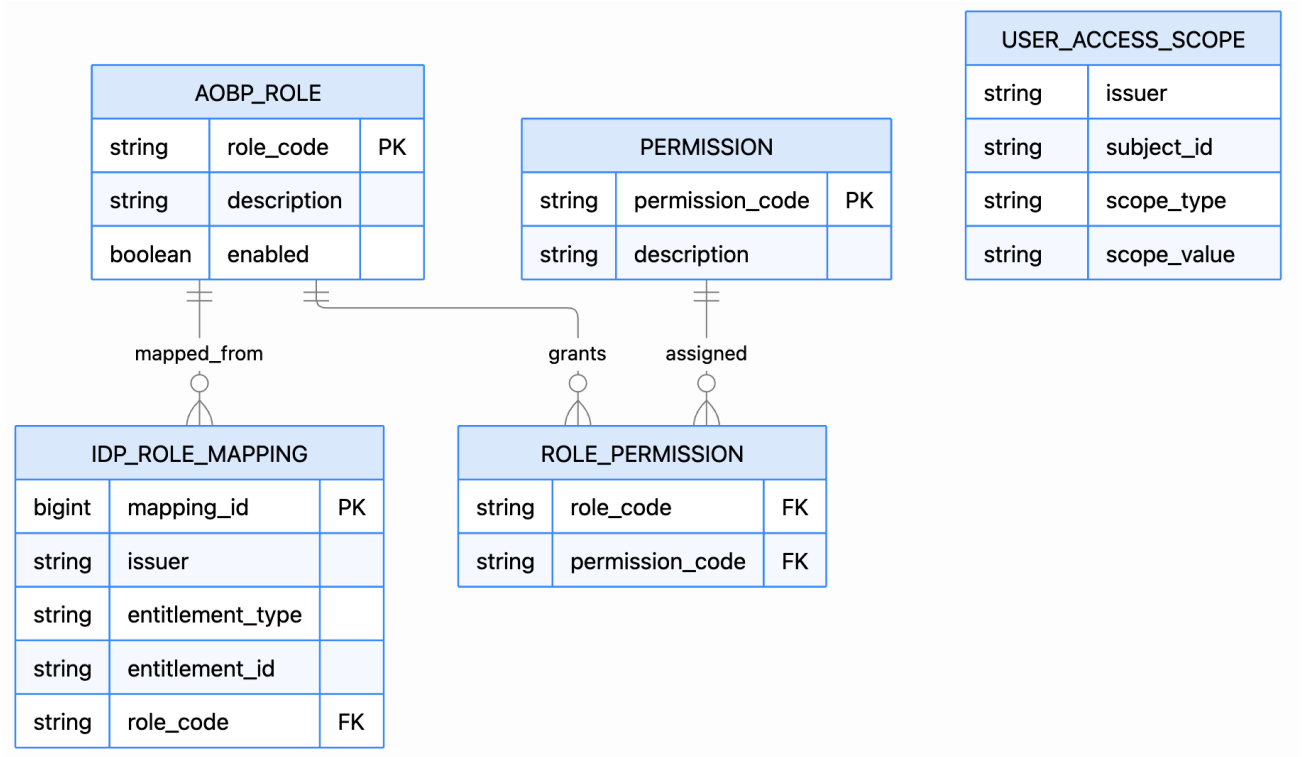
The authoritative terminal value must be read from business data, not from the browser request.

The Central Security Service resolves the user's permitted terminals and evaluates the terminal policy.



Authorization Database

Minimal schema


AOBP_ROLE
string role_code PK
string description
boolean enabled
PERMISSION
string permission_code PK
string description
USER_ACCESS_SCOPE
string issuer
string subject_id
string scope_type
string scope_value
mapped_from
grants
assigned
IDP_ROLE_MAPPING
bigint mapping_id PK
string issuer
string entitlement_type
string entitlement_id
string role_code FK
ROLE_PERMISSION
string role_code FK
string permission_code FK